



yes tell me

Yes — the main thing that likely changed the result is **target scaling plus the added global/distance features**. In practice, the second code should train more stably and capture long-range stress influence better than the first. [\[1\]](#) [\[2\]](#)

Most impactful changes

Change	Why it matters	Likely effect on R^2 / RMSE
Target normalization <code>von_mises/50\</code>	Reduces gradient scale and makes optimization easier	Usually the biggest training stability improvement
<code>dist_to_load</code> and <code>dist_to_bc</code> features	Give the model direct access to long-range physics cues	Often improves generalization more than deeper GNNs alone
Global context via <code>global_mean_pool</code>	Lets each node see a graph-level summary every layer	Helps when local message passing is not enough
LayerNorm + SiLU/AdamW	Stabilizes training and improves conditioning	Usually improves convergence consistency
Deeper network (8 vs 5 layers)	Increases receptive field and capacity	Helps only if optimization remains stable
CosineAnnealingLR	Smoothly reduces learning rate	Can improve final fit and reduce oscillation

What probably helped most

1. Target scaling.

This is the most direct reason the second model is easier to optimize, especially with Huber-style losses and gradient clipping.

2. Distance-to-load / distance-to-BC features.

For stress prediction in plates, these features are highly informative because stress is not only local around geometry; it is also driven by load transfer and support constraints.

3. Global context pooling.

This is a strong architectural upgrade because it injects graph-level information into every message-passing step.

What helped, but likely less than the above

- **LayerNorm**: improves stability, especially in deeper GNNs.
- **AdamW instead of Adam**: helps regularization.
- **Cosine annealing**: often helps final validation performance.
- **More layers**: useful only after the feature and optimization issues are solved.

Likely ranking by impact

Rank	Change
1	Target normalization
2	<code>dist_to_load+dist_to_bc</code>
3	Global context pooling
4	LayerNorm / better optimization setup
5	Deeper architecture
6	Scheduler / regularization tweaks

One important note: since the second code changes **multiple things at once**, you cannot attribute the gain to a single factor without ablation testing. The clean way is to test one change at a time against the same split and same seed.

Would you like me to design a compact **ablation table** for your exact setup so you can measure which change contributes most?

✱

1. <https://ludwig.guru/s/yes+tell+me>
2. <https://www.youtube.com/watch?v=EPL91FQyZmY>
3. <https://www.spanishdict.com/translate/'yes,tell me'>
4. <https://www.youtube.com/watch?v=Yd98t6wA15Y>
5. <https://www.ingles.com/traductor/'yes,tell me'>
6. https://www.youtube.com/watch?v=EDSRA_qGrFo
7. <https://www.youtube.com/watch?v=P9QhCDZS0cc>
8. <https://www.youtube.com/watch?v=eVgno1C6kio>
9. <https://gaana.com/song/yes-tell-me>
10. <https://www.instagram.com/popular/yes-tell-me-meaning-in-hindi/>